

Arquitectura RAG Completa

Módulo 7 · Sistemas RAG y Gestión del Conocimiento

Pipeline de ingesta y consulta, Adaptive RAG, CRAG, Agentic RAG, GraphRAG y checklist de producción.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Arquitectura RAG Completa

Construir un sistema RAG que funciona en un notebook es relativamente fácil. Construir uno que funciona en producción con alta calidad, baja latencia, múltiples usuarios, actualización continua del corpus, y monitorización de calidad es un proyecto de ingeniería serio. El salto del prototipo a producción para RAG se estima en semanas a meses de trabajo de ingeniería, y es uno de los puntos donde más proyectos de IA empresarial se atascan.

Este tema cubre la arquitectura completa de un sistema RAG de producción: todos los componentes, sus interfaces, los patrones de diseño establecidos, y las decisiones de arquitectura que más impactan en la calidad y la mantenibilidad del sistema a largo plazo.

Los componentes de un RAG de producción

Pipeline de ingesta (offline)

El pipeline de ingesta procesa los documentos fuente de forma asíncrona (batch o en streaming cuando llegan nuevos documentos). Sus etapas son: extracción de texto (parsing de PDF, DOCX, HTML, etc.), limpieza del texto (eliminar artefactos, normalizar formato), chunking (estrategia apropiada para el tipo de documento), enriquecimiento de metadatos (añadir categoría, fecha, autor, permisos), generación de embeddings (batch para eficiencia), y upsert en la base vectorial. El pipeline de ingesta debe ser idempotente (re-procesar el mismo documento produce el mismo resultado) y trackear qué documentos han sido procesados y cuándo.

Pipeline de consulta (online, tiempo real)

El pipeline de consulta se ejecuta para cada consulta del usuario en tiempo real. Sus etapas son: recepción de la consulta y autenticación del usuario, query transformation (opcional), embedding de la consulta, recuperación híbrida (dense + sparse, top-K), reranking (opcional, mejora precisión), construcción del prompt (combinar consulta + contexto recuperado + instrucciones), llamada al LLM, post-procesamiento de la respuesta (añadir citas, formatear), y logging de la sesión para monitorización.

Sistema de citas y trazabilidad

Un sistema RAG de producción debe siempre incluir citas que permitan al usuario verificar cada afirmación. Las citas incluyen: nombre del documento, número de sección o página, y fragmento exacto del texto recuperado. Esto tiene tres beneficios: permite verificar la corrección de la respuesta, construye confianza en el sistema, y facilita la auditoría. El prompt de generación debe instruir al LLM a citar explícitamente las fuentes de cada afirmación.

Patrones de arquitectura RAG avanzados

Adaptive RAG: decidir cuándo recuperar

Adaptive RAG (Jeong et al., 2024, KAIST) aprende a clasificar la complejidad de cada consulta y decide la estrategia de recuperación óptima: para preguntas simples de conocimiento general, el LLM responde sin recuperación (más rápido y barato). Para preguntas simples que requieren información específica, hace una sola recuperación. Para preguntas complejas que requieren múltiples fuentes, hace recuperación multi-hop. Esto reduce el coste y la latencia manteniendo la calidad.

CRAG: Corrective RAG con evaluación de los documentos recuperados

CRAG (Corrective RAG, Chen et al., 2024) añade un evaluador de relevancia que clasifica los documentos recuperados en Correct, Incorrect, o Ambiguous. Si los documentos son Incorrect o Ambiguous, CRAG activa una búsqueda web (o búsqueda adicional en el corpus) para obtener mejores fuentes. Esto reduce el riesgo de generar respuestas basadas en documentos recuperados pero irrelevantes.

Agentic RAG: recuperación multi-hop y planificación

Agentic RAG trata la recuperación como una acción de agente: el sistema puede decidir hacer múltiples recuperaciones secuenciales, donde cada recuperación usa la información de las anteriores para formular la siguiente consulta. Útil para preguntas que requieren encadenar múltiples pasos de razonamiento sobre información distribuida en varios documentos. Por ejemplo: "¿Qué impacto tuvo la adquisición de X sobre los ingresos de Y en el año posterior?"

SECCIÓN 4 · EJEMPLOS REALES

- RAG empresarial de conocimiento técnico (Salesforce Engineering): ingesta de 50.000 páginas de documentación técnica interna, wikis, y runbooks. Pipeline de ingesta con actualización incremental diaria. Búsqueda híbrida + reranking. Citas obligatorias en cada respuesta. Los ingenieros lo usan para encontrar respuestas a problemas técnicos sin necesidad de buscar manualmente en 50.000 páginas.
- Sistema de compliance regulatorio (banco europeo): RAG sobre el corpus completo de regulaciones aplicables (MiFID II, GDPR, CRR, directivas europeas actualizadas). Adaptive RAG con clasificación de complejidad: preguntas simples de definición responden rápido; preguntas sobre interacción entre regulaciones activan recuperación multi-hop. Citas a artículos específicos de la regulación.
- Knowledge assistant para gestión de propiedades (empresa inmobiliaria): RAG sobre contratos de arrendamiento, fichas técnicas de propiedades, regulaciones locales, y historial de mantenimiento. GraphRAG para preguntas sobre relaciones entre propietarios, inquilinos, y propiedades. Filtrado por metadatos de permisos (un gestor solo ve las propiedades de su cartera).

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: No separar los pipelines de ingesta y consulta. Mezclar el procesamiento de documentos con el servicio de consultas en el mismo proceso produce problemas de rendimiento (la ingesta consume recursos de CPU/GPU que compiten con las consultas en tiempo real) y dificultades operativas (no se puede actualizar el corpus sin afectar la

disponibilidad del servicio de consultas).

Error 2: No implementar control de acceso a nivel de retrieval. En sistemas multi-usuario, el filtrado de documentos por permisos de usuario debe aplicarse en la etapa de recuperación, no en la generación. Si se recuperan documentos confidenciales y luego se instruye al LLM "no menciones los documentos que el usuario no puede ver", el LLM puede ignorar la instrucción o filtrar de forma imperfecta.

Error 3: Contexto de ventana mal gestionado. Si $K=10$ chunks de 1000 tokens cada uno ocupan 10.000 tokens del contexto, y el LLM tiene una ventana de 16.000 tokens, apenas queda espacio para el system prompt, la consulta, y la respuesta. Gestiona activamente la suma de tokens del contexto recuperado y ajusta K o el tamaño del chunk para no saturar la ventana.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

El checklist de arquitectura para un RAG production-ready: pipeline de ingesta separado del servicio de consultas. Actualización incremental del corpus (sin re-indexar todo). Búsqueda híbrida (dense + sparse). Reranking. Control de acceso por metadatos. Citas obligatorias en las respuestas. Gestión del tamaño de contexto. Logging completo de queries, documentos recuperados, y respuestas. Evaluación continua con RAGAS. Monitorización de latencia y coste por consulta.

Para el diseño del prompt de generación en RAG: incluye las instrucciones de citar las fuentes, indica claramente que el modelo debe basar su respuesta solo en el contexto proporcionado, y añade la instrucción de decir "No encuentro información sobre esto en los documentos disponibles" cuando la respuesta no está en el contexto recuperado. Usa XML tags para separar el contexto de las instrucciones.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M7-T3 (Chunking): la estrategia de chunking determina la calidad del pipeline de ingesta.
- M7-T4 (Búsqueda híbrida): el componente de retrieval del pipeline de consulta.
- M10 (Agentes): Agentic RAG combina el patrón RAG con la arquitectura de agentes.

SECCIÓN 8 · RESUMEN EJECUTIVO

Un sistema RAG de producción tiene dos pipelines: ingesta (offline, procesar documentos → chunk → embed → indexar con metadatos) y consulta (online, query → embed → recuperar → reranquear → prompt → generar → citar). Los patrones avanzados son: Adaptive RAG (decide cuándo recuperar), CRAG (evalúa la calidad de lo recuperado), Agentic RAG (recuperación multi-hop), y GraphRAG (grafo de entidades en lugar de chunks). El checklist de producción incluye: búsqueda híbrida, reranking, control de acceso en retrieval, citas obligatorias, gestión de tokens de contexto, y monitorización continua. LlamaIndex es el framework especializado para casos de uso RAG complejos.